

Redis LangCache

Store LLM responses for AI apps in a semantic cache.

This is a snapshot of the Redis Docs from July 13, 2026.

See <https://redis.io/docs/latest/develop/ai/context-engine/langcache/> for the current version.

LangCache is currently available in preview. Features and behavior are subject to change.

Redis LangCache is a fully-managed semantic caching service that reduces large language model (LLM) costs and improves response times for AI applications.

LangCache overview

LangCache uses semantic caching to store and reuse previous LLM responses for repeated queries. Instead of calling the LLM for every request, LangCache checks if a similar response has already been generated and is stored in the cache. If a match is found, LangCache returns the cached response instantly, saving time and resources.

Imagine you're using an LLM to build an agent to answer questions about your company's products. Your users may ask questions like the following:

- "What are the features of Product A?"
- "Can you list the main features of Product A?"
- "Tell me about Product A's features."

These prompts may have slight variations, but they essentially ask the same question. LangCache can help you avoid calling the LLM for each of these prompts by caching the response to the first prompt and returning it for any similar prompts.

Using LangCache as a semantic caching service has the following benefits:

- **Lower LLM costs:** Reduce costly LLM calls by easily storing the most frequently-requested responses.
- **Faster AI app responses:** Get faster AI responses by retrieving previously-stored requests from memory.
- **Simpler deployments:** Access our managed service using a REST API with automated embedding generation, configurable controls, and no database management required.
- **Advanced cache management:** Manage data access, privacy, and eviction protocols. Monitor usage and cache hit rates.

LangCache works well for the following use cases:

- **AI assistants and chatbots:** Optimize conversational AI applications by caching common responses and reducing latency for frequently asked questions.
- **RAG applications:** Enhance retrieval-augmented generation performance by caching responses to similar queries, reducing both cost and response time.
- **AI agents:** Improve multi-step reasoning chains and agent workflows by caching intermediate results and common reasoning patterns.
- **AI gateways:** Integrate LangCache into centralized AI gateway services to manage and control LLM costs across multiple applications..

LLM cost reduction with LangCache

LangCache reduces your LLM costs by caching responses and avoiding repeated API calls. When a response is served from cache, you don't pay for output tokens. Input token costs are typically offset by embedding and storage costs.

For every cached response, you'll save the output token cost. To calculate your monthly savings with LangCache, you can use the following formula:

```
Est. monthly savings with LangCache =  
(Monthly output token costs) × (Cache hit rate)
```

The more requests you serve from LangCache, the more you save, because you're not paying to regenerate the output.

Here's an example:

- Monthly LLM spend: \$200
- Percentage of output tokens in your spend: 60%
- Cost of output tokens: $\$200 \times 60\% = \120
- Cache hit rate: 50%
- Estimated savings: $\$120 \times 50\% = \$60/\text{month}$



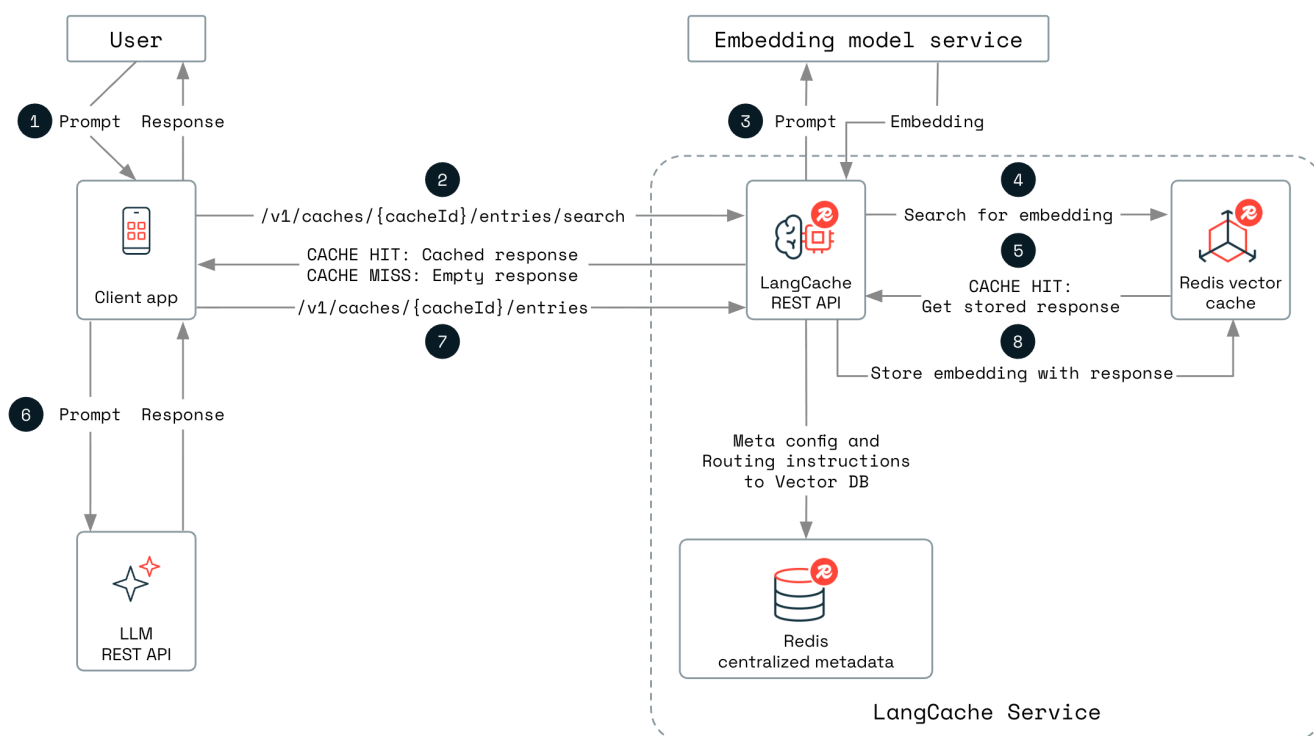
Note:

The formula and numbers above provide a rough estimate of your monthly savings. Actual savings will vary depending on your usage.

You can also use the [LangCache savings calculator](#) to estimate your annual savings with LangCache.

LangCache architecture

The following diagram displays how you can integrate LangCache into your GenAI app:



1. A user sends a prompt to your AI app.
2. Your app sends the prompt to LangCache through the POST `/v1/caches/{cacheId}/entries/search` endpoint.
3. LangCache calls an embedding model service to generate an embedding for the prompt.
4. LangCache searches the cache to see if a similar response already exists by matching the embeddings of the new query with the stored embeddings.
5. If a semantically similar entry is found (also known as a cache hit), LangCache gets the cached response and returns it to your app. Your app can then send the cached response back to the user.
6. If no match is found (also known as a cache miss), your app receives an empty response from LangCache. Your app then queries your chosen LLM to generate a new response.
7. Your app sends the prompt and the new response to LangCache through the POST `/v1/caches/{cacheId}/entries` endpoint.
8. LangCache stores the embedding with the new response in the cache for future use.

See the [LangCache API and SDK examples](#) for more information on how to use the LangCache API.

Get started

LangCache is currently in preview:

- Public preview on [Redis Cloud](#)
- Fully-managed [private preview](#)

Redis Cloud

Private preview

To set up LangCache on Redis Cloud:

1. [Create a database](#) on Redis Cloud.
2. [Create a LangCache service](#) for your database on Redis Cloud.
3. [Use the LangCache API](#) from your client app.

After you set up LangCache, you can [view and edit the cache](#) and [monitor the cache's performance](#).

See also our [Redis LangCache setup](#) tutorial video for advice on how to get started.